# Forest: Access-aware GPU UVM Management

Mao Lin
University of California, Merced
Merced, California, USA
mlin59@ucmerced.edu

Yuan Feng
University of California, Merced
Merced, California, USA
yfeng44@ucmerced.edu

Guilherme Cox
NVIDIA
Santa Clara, California, USA
gcox@nvidia.com

Hyeran Jeon
University of California, Merced
Merced, California, USA
hjeon7@ucmerced.edu

## Abstract

With GPU unified virtual memory (UVM), CPU and GPU can share a flat virtual address space. UVM enables the GPUs to utilize the larger CPU system memory as an expanded memory space. However, UVM's on-demand page migration is accompanied by expensive page fault handling overhead. To mitigate such overhead, tree-based neighboring prefetcher (TBNp) has been used by GPUs. TBNp effectively reduces page faults by exploiting locality at multiple levels. However, we observe its access-pattern oblivious design leads to excessive page thrashing and unnecessary migrations. In this paper, we tackle the inefficiencies with a novel access-aware UVM management, *Forest*. Forest uses a software-hardware codesign to configure the optimal tree prefetchers at runtime based on each data object's access patterns. With the heterogeneous tree-based prefetching, Forest provides 1.86× and 1.39× speedups over the baseline TBNp and state-of-the-art optimization solutions, respectively. Forest also shows a 1.51× speedup for real-world deep learning models, including CNNs and Transformers.

## CCS Concepts

• **Computer systems organization** → **Parallel architectures**; • **General and reference** → **Performance**; **Design**.

## Keywords

Unified Virtual Memory, Prefetching, GPU, Memory Management

**ACM Reference Format:**
Mao Lin, Yuan Feng, Guilherme Cox, and Hyeran Jeon. 2025. Forest: Access-aware GPU UVM Management. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25), June 21–25, 2025, Tokyo, Japan.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3695053.3731047

## 1 Introduction

GPU unified virtual memory (UVM) has gained popularity thanks to its hassle-less memory expansion capability. With UVM, data objects can be placed anywhere in either CPU or GPU memories and can be fetched to the GPU memory when GPU accesses them frequently [4, 16]. In GPU computing, the memory capacity is one of the main performance limiters [22, 23, 28, 47, 53]. Given the increasing size of emerging workloads, UVM is one of the most favorable technologies because it allows deploying such big data workloads on GPUs. Alongside the rapid evolution of GPU architectures, UVM has been one of the core mechanisms to effectively expand GPU memory capacity through interplay with emerging technologies such as cache-coherent and access counter-based data sharing [17, 18].

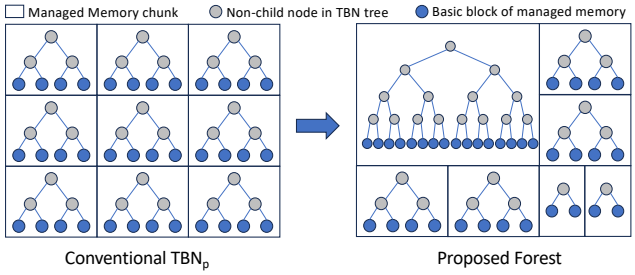



**Figure 1: TBNp vs. Forest: TBNp uses homogeneous tree prefetchers for all memory blocks while Forest flexibly applies heterogeneous trees to support diverse access patterns.**

To better utilize the limited GPU memory space with proactive page migrations, several solutions have been proposed, such as prefetching and pre-eviction methods [11, 26, 27, 50], address translation optimization [41–43], page thrashing reduction [44, 45] and multi-path data transfer [12]. One industry solution used by several generations of NVIDIA GPUs is tree-based neighboring prefetcher (TBNp) [2, 26]. It decides the target pages to prefetch based on access locality and controls the prefetch aggressiveness with tree-based prediction. TBNp partitions memory space into 2MB virtual address blocks (VABlocks) and manages each VABlock with a full binary tree structure. A 64KB leaf node is the minimum migration unit.

Though the TBNp effectively exploits access locality, we observe two limitations: 1) one tree configuration is used for all applications and all data objects regardless of the memory access patterns and 2) the UVM driver decides page eviction based on the page fault events, not by the actual page accesses. Both limitations stem from the driver-side UVM management. As the GPU driver cannot monitor the memory access patterns in the GPU device, TBNp often

This work is licensed under a Creative Commons Attribution 4.0 International License.
*ISCA '25, Tokyo, Japan*
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1261-6/25/06
https://doi.org/10.1145/3695053.3731047

produces suboptimal migration decisions, which result in severe page thrashing and unnecessary migrations.

To resolve the inefficiencies, we propose an access-aware GPU UVM management, namely *Forest*. Forest manages UVM in the unit of individual data objects and configures a prefetch tree that works best for each data object. Figure 1 shows the conceptual differences between TBNp and Forest. TBNp manages UVM in fixed-size memory units with homogeneous prefetch trees (e.g., one tree manages a 2MB memory space with 64KB leaf nodes). Forest on the other hand manages UVM in the unit of individual data object's size and configures one tree for each data object that works the best for the data object (i.e., heterogeneous prefetch trees).

There are four challenges to managing the UVM with access-aware heterogeneous tree structures. **First, the access pattern of individual data objects should be identified.** The current UVM is managed purely by the driver located in the host CPU [2]. Therefore, the UVM can detect access patterns only in the granularity of page fault, which is too coarse-grained. The page access counters used by some GPUs only reflect page hotness, which can not be used for access pattern detection. To enable more fine-grained and correct access pattern detection, Forest repurposes the existing hardware page access counters to record the page access sequence within each data object, with a lightweight hardware support. By leveraging the existing access counter fetch function, the UVM driver can detect access patterns of each object. **Second, the prefetcher should be configured at runtime.** The TBNp information is managed by the UVM driver. We extend the existing UVM driver to integrate two 1-bit metadata per non-leaf node, with which Forest dynamically merges the leaf nodes or isolates the TBNp tree. **Third, optimal tree configuration should be identified.** From rigorous analysis, we introduce four access patterns that significantly influence prefetch performance and propose the best-performing tree configuration for each pattern. **Fourth, the access pattern detection should not cause performance overhead.** To reduce the burden of runtime access pattern detection, we propose *speculative Forest (SpecForest)*. The SpecForest uses compiler analysis and pattern recording to statically detect simple access patterns and reuse detected patterns for subsequent kernels without runtime pattern detection. For complex access patterns (e.g., indirect accesses), we propose a novel *pattern similarity detection*. At compile time, we group the data objects that use similar data indexing. At runtime, we assign the same tree prefetcher to all objects in the same group as soon as one data in the group finishes the pattern detection.

Our evaluations under a 150% memory oversubscription show that Forest and SpecForest outperform the Baseline TBNp by 72% and 86%, respectively. Compared to state-of-the-art solutions [26, 27, 29], Forest and SpecForest respectively achieve 28% and 39% speedups. Additionally, SpecForest delivers an average performance improvement of 51% across various real-world deep learning workloads, including CNN-based and Transformer-based models.

Our contributions are as follows:

- To the best of our knowledge, this is the first study that uncovers the intrinsic issue of unnecessary page migrations and thrashings of UVM TBNp.

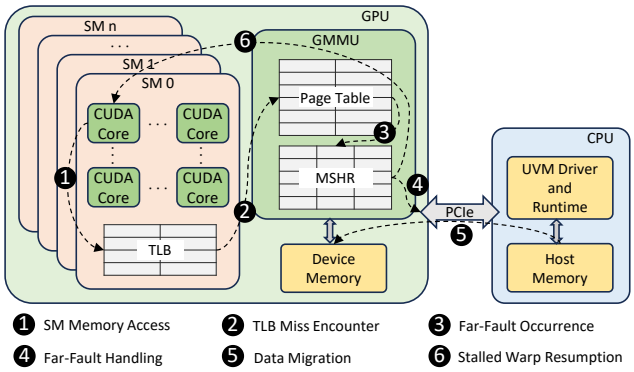



**Figure 2: UVM walkthrough for far-faults.**

- We propose *Forest*, an access-aware UVM management. Forest uses software-hardware co-design to fundamentally tackle the root cause of the inefficiencies of software-based UVM management. Forest introduces a novel heterogeneous prefetching strategy while existing studies mostly focused on migration threshold adjustment over the existing homogeneous tree prefetcher [25–27, 29].
- We introduce compiler-assisted optimized access pattern detection, namely *SpecForest*, which further improves performance with early adoption of optimal prefetcher.
- We demonstrate the effectiveness of Forest across an extensive number of workloads, including general-purpose GPU benchmarks and real-world deep learning (DL) applications (e.g., Transformers and CNNs).

## 2 Background and Related Work

### 2.1 GPU Unified Virtual Memory System

UVM allows CPUs and GPUs to share a flat virtual memory system. With UVM, CPUs and GPUs can communicate via zero-copy pointer sharing and seamless data transfer without explicit memory copies. If a data object is allocated by GPU applications (e.g., through `cudaMallocManaged`), the data pages are copied to the GPU memory when the data is accessed by the GPU kernel. The UVM driver handles such implicit memory copy through *far-fault* handling. Far-faults occur when pages are not physically present in the GPU device memory [2, 3, 25–27]. The far-faults trigger on-demand data allocation and migration. When a GPU kernel executes a memory operation on the data if the GPU-side MMU (GMMU) finds the data page is not in the GPU memory, a far-fault is triggered. Then, the UVM driver copies the page from CPU to GPU. Figure 2 illustrates the end-to-end steps of the far-fault handling. Despite the advantages of UVM, the seamless page sharing does not directly lead to a performance gain because the long far-fault handling latency is added to the critical path of memory operations, as shown in the figure [25–27, 38, 46].

With UVM, the limited GPU memory space can be effectively expanded to the CPU system memory and can run an application with a larger memory footprint than the GPU memory (i.e., *memory oversubscription*). To support memory oversubscription, UVM also handles runtime page eviction. If GPU doesn't have space to fetch a

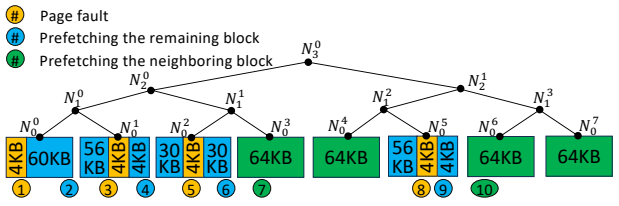



**Figure 3: TBNp operations. For simplicity, this example denotes 512 KB of consecutive memory, while each TBN tree represents a 2 MB VABlock in the original TBNp.**

new page, the least recently used (LRU) 2MB virtual address block (VABlock) is evicted from GPU to CPU. The UVM driver identifies the LRU pages by monitoring the sequence of far-faults. Thus, the pages that are least recently copied to the GPU are selected to be evicted [2]. Because evictions do not consider GPU-side access recency, hot pages are often incorrectly victimized [27].

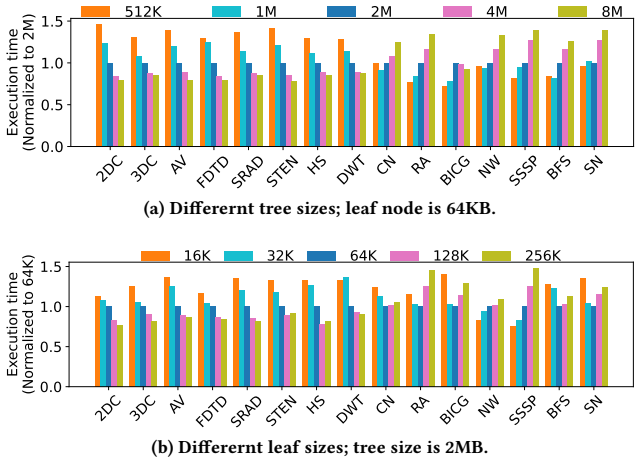
## 2.2 Tree-based Neighboring Prefetcher (TBNp)

To mitigate the overhead associated with UVM, NVIDIA GPUs employ TBNp, which uses a full binary tree structure [2, 25, 26]. Each 2MB VABlock is managed with a five-level full binary tree having 32 64KB leaf nodes. Each non-leaf node holds two metadata, namely $\mathcal{N}_{\text{total}}$ and $\mathcal{N}_{\text{migrated}}$, each indicating the node's total and migrated child node size. TBNp manages migrations in the unit of a leaf node. Therefore, once a page encounters a far-fault, the entire 64KB in the associated leaf node is migrated. TBNp also does *proactive* prefetching leveraging the tree structure. In each 2MB tree, if more than 50% of the leaf nodes are migrated to the GPU memory, the TBNp proactively prefetches the remaining leaf nodes to the GPU by assuming a high locality. With these two prefetch algorithms, multiple on-demand far-faults can be skipped with one far-fault handling.

Figure 3 illustrates the operations of the TBNp. Each box attached to the leaf node is a 64KB basic block. ① A far-fault occurs on the leftmost yellow 4KB page within $N_0^0$. Then, the GPU driver migrates the page to the GPU. TBNp then locates the leaf node holding the page. ② TBNp prefetches the remaining pages (blue) in the same leaf node to the GPU, as the TBNp's migration unit is 64KB. This prefetching process is applied to subsequent faults in $N_0^1$ (③ and ④) and $N_0^2$ (⑤ and ⑥). After migrating these three 64KB blocks, the prefetcher notices that over 50% of the pages within node $N_2^0$ are in the GPU memory (i.e., $\mathcal{N}_{\text{migrated}}$ is greater than 50% of $\mathcal{N}_{\text{total}}$ in $N_2^0$). ⑦ Then, the prefetcher migrates all remaining pages (green) of $N_2^0$ to the GPU memory. ⑧ The next far-fault occurs in $N_0^5$ and ⑨ a corresponding 64KB region is also migrated. ⑩ UVM observes that over 50% of the pages in node $N_3^0$ are in the GPU memory and the prefetcher migrates all remaining pages in $N_3^0$ to the GPU memory (i.e., $\mathcal{N}_{\text{migrated}}$ is greater than 50% of $\mathcal{N}_{\text{total}}$ in $N_3^0$).

## 2.3 Other UVM Prefetching Solutions

Besides TBNp, several studies have aimed to reduce the far-fault overhead. Choi et al. [12] introduced a multi-path migration strategy in multi-GPU environments that bypasses eviction during




**(a) Differernt tree sizes; leaf node is 64KB.**

**(b) Differernt leaf sizes; tree size is 2MB.**

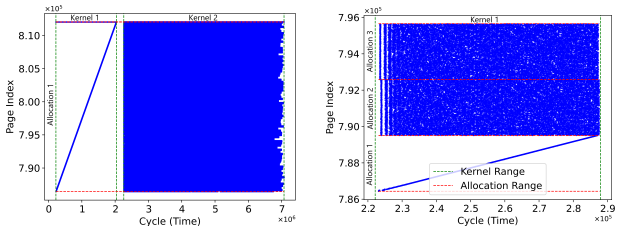
**Figure 4: Execution time with diverse tree configurations (lower is better).**

critical far-fault handling; Kim et al. [38] used a batched far-fault handling, thus amortizing the time spent on GPU runtime fault handling. Other studies developed smarter algorithms for data prefetching or eviction under memory oversubscription, such as DeepUM [35], which utilized the repetitive nature of kernel executions and memory access patterns in DL training to prefetch data based on historical behaviors. Several studies tackled the limitations of TBNp. Ganguly et al. proposed a tree-based eviction and access-counter-based dynamic memory management strategies in their series of papers [25–27]. Early-Adaptor [29] designed a model that automatically adjusts prefetching aggressiveness based on page fault history. These prior works overlooked the fundamental limitations of software-only UVM design, which Forest aims to address via software-hardware co-design.

# 3 Motivation

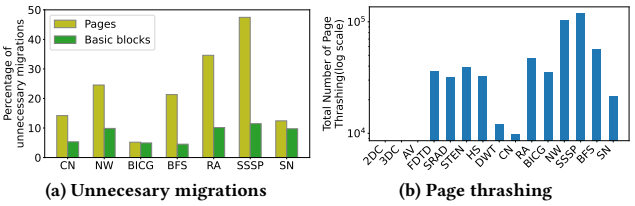
## 3.1 One Configuration Doesn't Fit All

To understand the effectiveness of the TBNp for improving performance, we measured the execution time while varying the tree configurations. We used a cycle-level GPU architecture simulator that implements TBNp [48]. The tested applications are selected from various GPGPU benchmark suites. More details are in Table 1 and Table 2. Figure 4 shows the normalized execution time when using different tree configurations. Both figures use the conventional TBNp size as the baseline, where each tree prefetcher manages a 2MB memory space with 64KB leaf nodes. From these results, we couldn't find one optimal configuration that performs the best for all applications. In Figure 4a, the first eight applications show better performance with larger trees, while the remaining applications do not. RA and SSSP show better performance with smaller trees, whereas CN, NW and BFS perform better with medium (1MB) trees. Likewise, in Figure 4b, the first eight applications perform better with larger leaf nodes, while SSSP and NW show better performance with smaller leaf nodes. We couldn't find a clear trend for CN, RA and BICG. Even in the first eight applications, some applications




(a) BICG (b) RA

**Figure 5: Memory access patterns of BICG and RA.**




(a) Unnecesary migrations (b) Page thrashing

**Figure 6: Side-effects of TBNp**

(STEN and HS) perform better with a 128KB leaf node, which is not the largest leaf node size. This result raises strong doubt about the effectiveness of homogeneous TBNp tree configuration. It is noteworthy that none of the 15 applications perform the best with the baseline configuration. Given that the baseline tree configuration is used as the default setting in the commercialized GPUs as well as academic studies, we believe that the UVM setting should be fundamentally revisited.

**We know the application's memory access pattern. Can we find the best tree configuration for the application?:** No. In more than half of our tested applications, we observed that there is no such tree configuration that is best for all kernels executed by the application. Even in each kernel, the access patterns are significantly different among data objects. Furthermore, the same data objects are often accessed differently in different kernels.

Figure 5 shows such a diverse example. Each graph is partitioned vertically for multiple kernels (i.e., in BICG, one data object is shared by two kernels) and horizontally for multiple objects (i.e., RA has three data objects in one kernel). For each kernel and data object, the graphs show the page addresses (Y-axis) accessed throughout the execution time (X-axis). In BICG, the graph shows how the two kernels access the same data differently. In kernel 1, data has a clear linear access pattern. However, in kernel 2, almost all the pages are accessed from the beginning without a certain pattern. This is because the data is used as output and input in kernels 1 and 2, respectively. Kernel 1 generates the data and shares the pointer as an input with kernel 2. Due to the limited coherence and synchronization supports, GPU applications typically have many readers and a single writer for each data. Each warp executes in an in-order fashion, which is naturally synchronized. Thus, the output data tends to be accessed linearly, as shown in kernel 1. On the other hand, input data doesn't need to be synchronously accessed. If the data is read by multiple warps in multiple SMs, the access pattern can be easily scattered, as shown in kernel 2. In RA, in one kernel, the top two data show similarly scattered access patterns while the bottom data has a clear linear pattern. Thus, we cannot summarize each application's access pattern with one pattern.

For the applications having diverse access patterns across kernels and data objects, one prefetch configuration cannot produce the best performance. Some studies [10, 27, 29] tackled such access pattern diversity by adapting migration thresholds. While these solutions can dynamically control the migration aggressiveness, they have limited impact because the thresholds are not tailored to individual data and the aggressiveness is bounded to the default TBNp tree size. We propose a more fundamental solution, which dynamically configures the TBNp for individual data objects.

## 3.2 TBNp, A Hidden Source of Memory Oversubscription.

We also examined the effectiveness of TBNp on memory utilization. Two important performance metrics of memory prefetchers are accuracy and timeliness. If data are incorrectly prefetched or prefetched too early or late, the prefetched data will end up unused while unnecessarily occupying memory space. We examined if all the migrated data are accessed by the GPU program before being evicted. We define *unnecessary migrations* as the number of migrations that end up having zero accesses until eviction.

Figure 6a shows the six applications that encountered a significant number of unnecessary migrations; from 5% to 48% of the total migrated pages were unnecessarily migrated. We also measured the unnecessary migrations in 64KB basic block (leaf node) units. The unnecessary block migrations occur when blocks are prefetched alongside their neighboring blocks (e.g., green blocks in Figure 3) but remain unaccessed until they are evicted. BICG and SN show a similar ratios in block and page units, which indicates that most of the needless migrations stem from the unnecessary blocks within the large tree. In other words, many of the blocks migrated by the proactive prefetch are not accessed, while the blocks that are migrated due to far-fault are well utilized. Conversely, the other applications show a significantly higher percentage of unnecessary pages compared to blocks. This is because these applications sparsely access only a few pages within each migrated block. In other words, while most of the migrated blocks are accessed at least once, many pages within each block are never accessed. This is mainly caused by the large migration unit; 64KB minimum regardless the number of pages accessed within the block. In both cases, the uniform tree configuration is the root cause of the unnecessary migrations.

## 3.3 Driver-driven Page Eviction and Thrashing

To evict pages under oversubscription, UVM driver maintains an LRU list. The LRU list is updated upon page faults. Thus, the LRU list does not accurately reflect the actual access recency on device. Note that even under access-counter-based UVM migration, the eviction policy still uses the far-fault-based LRU list. To understand the impact of this driver-driven eviction, we measured page thrashing, which is defined as the number of pages that are migrated again after eviction. Figure 6b shows the results when applications use 150% larger memory than the GPU memory. In the benchmarks with

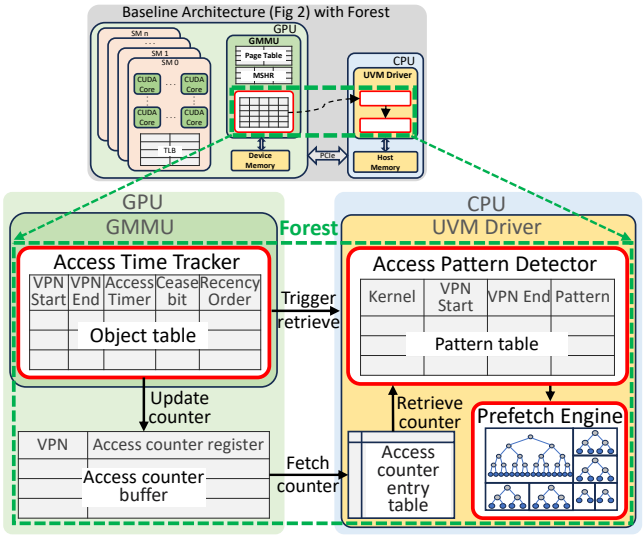



**Figure 7: Architectural design of Forest (Newly added or modified components are highlighted with red outlines).**

linear access patterns with limited data reuse (e.g., 2DC, 3DC, and AV), there was no page thrashing because the migration sequence and LRU order match naturally. For the remaining benchmarks, from 9K to over 100K pages were remigrated after eviction. For some applications, the total thrashed pages (counting repetitive thrashing separately) is worth over 5.7× of their memory footprint! Such severe page thrashing occurs due to three reasons: 1) insufficient GPU memory, 2) suboptimal prefetcher leading to unnecessary migrations, and 3) driver-driven LRU decision evicting data soon to be migrated again. From our experiments, the suboptimal prefetcher and driver-driven eviction contribute an average of 27% and 6% (up to 46% and 18%) of the thrashings, respectively. This motivates us to reconsider the eviction mechanism for better memory utilization.

## 4 Forest: Access-aware GPU UVM Management

To tackle the inefficiencies of the access-oblivious homogeneous TBNp, we propose an access-aware GPU UVM management, *Forest*. Forest dynamically detects access patterns of individual data objects and configures the prefetch tree for each data object.

### 4.1 Overall Design

Figure 7 shows the overall architecture of Forest, which consists of three main components: an *access time tracker (ATT)*, an *access pattern detector (APD)*, and a *prefetch engine (PE)*. ATT is a newly added on-device page access timing tracker, which is integrated in the GMMU. ATT uses the existing hardware page access counters to record the sequence of page accesses for each UVM-managed data object. APD is a new UVM driver module, which collects the page access timing information recorded by ATT once per the pre-defined profiling phase, determines the access pattern, and recommends the proper TBNp for each data object. PE in the existing UVM driver operates migrations and prefetches. We extend the PE to configure the recommended TBNp configuration and operate migrations and prefetches accordingly.

### 4.2 Access Time Tracker (ATT)

To understand the access patterns of individual data objects, we should be able to collect the page access recency. We could leverage existing hardware page access counter registers [27, 56], which monitor the number of accesses per page. However, the existing hardware page access counters reflect each page's access intensity rather than access recency. ATT repurposes the access counter registers to collect page access recency with the help of an *object table*, as illustrated in Figure 7.

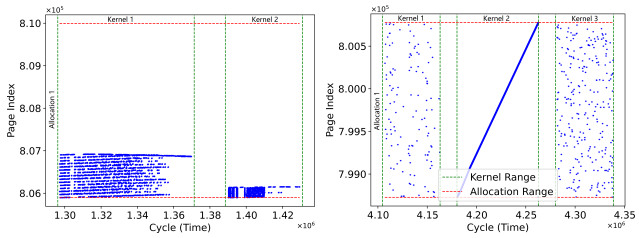
Each entry of the object table is associated with one data object managed by UVM and consists of a start and an end virtual page numbers (VPN), an access timer, a recency order, and a cease bit. Each entry is reserved when a data is allocated with a UVM malloc API (e.g., `cudaMallocManaged`) and is filled upon kernel invocation.

The `access timer` maintains the total page access count of the data object. Upon a page access, the access timer value of the associated data is incremented by one and the updated value is written to the page's access counter register. This way, the hardware page access counter registers can reflect the access order of each page within the data object. For example, suppose that a data object has a linear access pattern, so the pages `N`, `N+1`, and `N+2` have their hardware access counter register values `C`, `C+1`, and `C+2`, respectively. This reflects that consecutive pages are accessed in subsequent timings. The access timer in ATT has the latest access counter register value, `C+2`. If another page is accessed, the access timer will be incremented to `C+3` and the new page will have `C+3` as its hardware page access counter register value.

The existing page access counters are implemented with 32-bit registers and are updated by the GMMU at every local memory access during the TLB lookup [27, 56]. We leverage the existing counter architecture and operation, except for the value of the access counter registers. The access timer in the object table and the page access counter registers provide object-level access intensity and page-level spatial locality, which together can support more accurate migration decisions.

The `recency order` is the LRU list of the objects in the object table, which will be used for page replacement (details in Section 4.5). The `cease bit` is a 1-bit indicator that shows whether an access pattern has been detected by APD. APD detects the data object's access pattern by using the access timing information collected by ATT (details in Section 4.3). Once the APD identifies the access pattern, the cease bit is set to 1 and the ATT stops the monitoring.

The access timer has 32 bits. With a 32-bit access timer, up to 4 billion accesses can be recorded, which is rarely overflown according to our experiments. However, if the access timer reaches the maximum value, the access timer as well as the page access counter registers associated with the data are halved. According to our analysis of GPU benchmark applications, each application uses up to 10 UVM data objects. Real-world applications (e.g., open-source deep learning and large language models) also rarely use more than eight UVM data per kernel [9, 21]. Thus, we use a 10-entry object table. If a kernel uses more than 10 UVM objects, the GPU driver selects the largest 10 objects. The smaller objects with less performance impact are supported by the default tree configuration instead of Forest. The recency order has 4 bits to represent the LRU order among the 10 objects. The `VPN start`




(a) DWT (b) SSSP

**Figure 8: Memory access patterns of `DWT` and `SSSP`.**

and end fields have 40 bits each [24] and the cease bit is 1 bit. Thus, the object table has a total of 147 bytes. To support multi-kernel execution, we employ as many object tables as the number of maximum concurrent kernels. GPUs support 16 to 128 concurrent kernels [19] so object tables can be 18.375KB at the maximum.

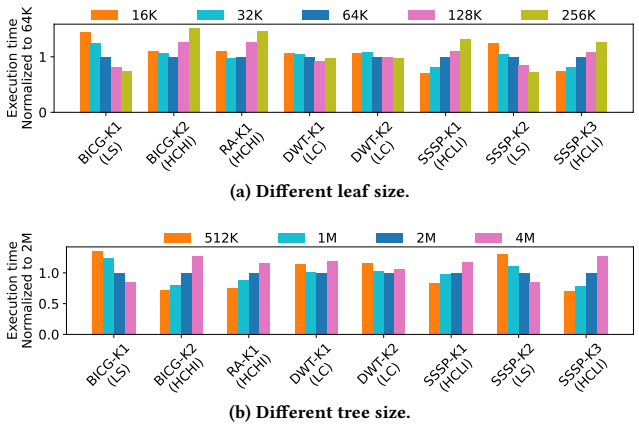
## 4.3 Access Pattern Detector (APD)

APD detects access patterns of individual data objects and identifies the optimal TBNp configuration.

*4.3.1* ***Access timer-based pattern monitoring.*** To detect access patterns, Forest leverages ATT's page access timing information. To copy the access timer information of the target object from GPU to APD, we leverage an existing UVM driver function (e.g., `fetch_access_counter_buffer_entries(.)`) which copies all access counters [51]. We extend this function to copy the counters of target objects. The access counters are copied from the GPU to APD once per a pre-defined number of data accesses (10K accesses in our design). If ATT finds that an object's access timer reaches the threshold accesses, it triggers an interrupt. Then, the UVM driver executes the aforementioned extended function and reads access counters of all pages within the data object.

The existing UVM driver copies access counters via the access counter buffer data structure as illustrated in Figure 7. We use the existing architectures. Out of the parameters used in the buffer, APD focuses on the VPN and access counter of each page, denoted as $P_{Set}$ (= a set of $(p_i, t_i)$ pairs, where $p_i$ and $t_i$ are the page's VPN and hardware access counter register value, respecitvely). APD examines $P_{Set}$ of each data object and performs a pattern check based on the pre-defined access patterns, which will be explained in Section 4.3.2. If one of the patterns is detected, the profiling for the data object concludes. The APD then instructs the PE to update the TBNp configuration for the data object, records the pattern, and notifies the ATT to cease triggering interrupts by setting the cease bit in the object table. Note that APD is a UVM driver module which can be operated without halting GPU execution. During the pattern detection and TBNp configuration, the GPU continues the execution with the default TBNp configuration.

The patterns are recorded in a *pattern table*, as shown in Figure 7. Each entry of the pattern table is associated with a UVM-managed object and consists of kernel ID, the start and end VPNs, and pattern fields. The pattern field has two bits because we use four patterns, which will be described in the next section. Similar to the object table in ATT, each entry of the pattern table is reserved when a data is allocated with a UVM malloc API. If no pattern is detected, the APD repeats the procedure after the next monitoring phase. To prevent continuous monitoring throughout the entire execution, after a pre-defined (10 in our design) rounds of monitoring, the APD designates the data object as having a default access pattern and pauses further profiling for that data. The default access pattern will be explained shortly.




(a) Different leaf size.

(b) Different tree size.

**Figure 9: Performance impact of tree configurations for different access patterns (lower is better).**

*4.3.2* ***Access pattern classification & tree configuration.*** Access pattern consideration for UVM optimization is not new. Most studies [10, 25, 29] classified the access pattern of each application into either regular or irregular. However, the access regularity cannot reflect access aggressiveness, which is essential for designing a prefetch algorithm. Thus, we propose more fine-grained classifications. We identified four access patterns that influence the performance of prefetch algorithms the most by analyzing 91 data objects used by popular GPU benchmark suites (Table 1): *Linear/Streaming*, *Non-Linear High-Coverage High-Intensity*, *Non-Linear High-Coverage Low-Intensity*, and *Non-Linear Low Coverage*. Our proposed classification can support both access locality and aggressiveness. From the analysis, we also identified the best-performing prefetch tree per access pattern. We show sample results (Figure 9) while explaining our tree selection below.

*Linear/Streaming (LS):*

In LS pattern, the pages of the data object are accessed in either forward or inverse order. In GPU, due to concurrent memory accesses by warps and SMs, individual pages may not be accessed in clearly linear timings. Thus, we check if the accesses follow approximate linearity by using a linear regression algorithm. We first perform a linear regression [33] on the $P_{Set}$ of each object to model the relationship between the pages ($p_i$) and their access timing information ($t_i$). We then calculate the coefficient of determination ($R^2$) [7], a common metric used to evaluate the degree of linearity. If $R^2$ is closer to 1, the dataset follows a stronger linear pattern. The access pattern is classified as LS if the profiling data satisfies the following condition:

$$R^2_{P_{Set}} > R^2_{LS} \text{ [7]} \quad (1)$$

where $R^2_{P_{Set}}$ is the calculated coefficient of profiling data and $R^2_{LS}$ represents the linearity threshold (set to 0.8 in this study). Kernel 1 in Figure 5a is an example LS pattern. In the LS pattern, the accessed pages are rarely reused within the kernel. Therefore, a more aggressive prefetching and eviction is favorable. Under TBNp's 50% proactive prefetching policy, more pages will be proactively prefetched if a larger tree is used. As each tree covers consecutive VPNs, the proactively prefetched pages will likely be accessed by the LS data objects. The results in Figure 9 align with our design; the objects with LS access pattern perform better with larger tree and leaf nodes. In our experiments, we use a 4MB tree with 256KB leaf nodes for LS data, which are the maximum sizes that we considered. Note that we consider trees between 512KB and 4MB and leaf nodes between 16KB and 256KB.

*Non-Linear High-Coverage High-Intensity (HCHI):*

$$p_n - p_1 \geq N_p \times \mathcal{P} \text{ and } N_a \geq N_p \times \mathcal{A} \quad (2)$$

where $p_1$ and $p_n$ are the smallest and largest VPNs of the target object accessed by the GPU during the profiling phase, $N_p$ represents the total number of pages in the data object, $N_a$ denotes the number of accessed pages, $\mathcal{P}$ indicates the threshold of access coverage (0.6 in our design), and $\mathcal{A}$ represents the threshold of access intensity (0.4 in our design). This pattern means that the accessed VPN range and the accessed page count within the monitoring period are larger than the pre-defined portion of the pages in the data. In other words, a large number of pages in wide address ranges are accessed for a short time. Kernel 2 in Figure 5a is an example. For such scattered accesses, aggressive prefetching easily leads to unnecessary migrations. Because HCHI data accesses a large number of pages, while only few data within the pages are actually accessed, if we use a large tree, the GPU memory will be quickly filled with a significant amount of unused data. Therefore, we configure HCHI to use small trees so that prefetches can happen within the small tree boundary. The leaf node is configured to be default size because small leaf nodes will lead to frequent page faults while too large leaf nodes will make the small trees quickly fill up. The results in Figure 9 support our design; HCHI data show better performance with smaller tree and medium size leaf nodes. In our experiments, we use a 512KB tree with 64KB leaf nodes.

*Non-Linear High-Coverage Low-Intensity (HCLI):*

$$p_n - p_1 \geq N_p \times \mathcal{P} \text{ and } N_a < N_p \times \mathcal{A} \quad (3)$$

This pattern means that a wide address range within the target data object is accessed with a small number of pages during a monitoring period. Kernels 1 and 3 in Figure 8b are examples. Similar to the HCHI, the HCLI pattern is better to use less aggressive prefetch to reduce unnecessary migrations. Compared to HCHI, HCLI shows more severe access sparsity. Therefore, we configure both trees and leaf nodes small. The results in Figure 9 justify our design as the HCLI data show better performance with smaller tree and leaf nodes. In our experiments, we set a 512KB tree with 16KB leaf nodes.

*Non-Linear Low Coverage (LC):*

$$p_n - p_1 < N_p \times \mathcal{P} \quad (4)$$

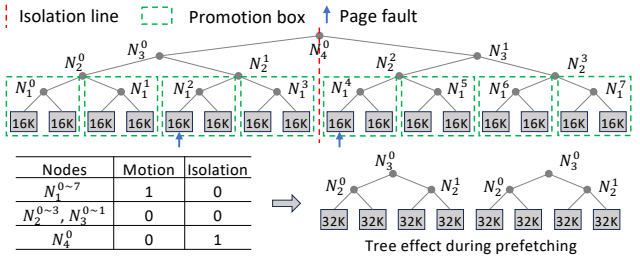



**Figure 10: Example of motion bit and isolation bit.**

This class includes objects with small footprints and those not settled to a clear access pattern. Figure 8a shows an example. For this pattern, we use the default tree configuration (2MB tree with 64KB leaf nodes) because LC data neither shows a strong access trend nor has clear performance impacts with different tree configurations. LC patterns in Figure 9 also do not show a clear pattern. We use LC as a default pattern.

## 4.4 Prefetch Engine (PE)

PE is in the existing UVM driver and performs migrations and prefetches. We extend the PE to reconfigure TBNp for individual data objects. As described in Section 2.2, the baseline TBNp uses metadata, $\mathcal{N}_{\text{migrated}}$ and $\mathcal{N}_{\text{total}}$, per non-leaf node to control migrations. For dynamic tree configuration, we add two 1-bit metadata per non-leaf node; an `isolation` bit and a `motion` bit.

The `isolation` bit is used to change the tree size. The `motion` bit controls the leaf size. The `isolation` and `motion` bits cannot both be set to 1 in the same node. 16KB is our minimum configurable leaf size. Thus, initially, each UVM data is partitioned into 16KB units and the `isolation` and `motion` bits are configured to represent the default 2MB tree with 64KB leaf nodes. If the `isolation` bit is set to 1, PE isolates the left and right child nodes, preventing them from influencing each other's migration behaviors. Otherwise, all child nodes and the parent node itself are considered part of the same TBN tree.

In Figure 10, the `isolation` bit of $N_4^0$ is 1 to indicate that the left and right trees are independent. In other words, the figure represents two prefetch trees each covering 128KB (=8 × 16KB) memory space. When the `motion` bit is set to 1, the PE promotes all child nodes to a single basic block. If it is not set, PE demotes the child nodes to separate nodes. In Figure 10, the `motion` bits of the immediate parent nodes of all leaf nodes ($N_1^0$ to $N_1^7$) are 1, which indicates that the trees use 32 KB basic blocks (or leaf nodes).

Once the tree is configured with these `motion` and `isolation` bits according to APD's recommended tree architecture, PE performs prefetching based on the values of the existing metadata, $\mathcal{N}_{\text{migrated}}$ and $\mathcal{N}_{\text{total}}$ of the updated tree. For example, in Figure 10, after all nodes under $N_2^0$ are migrated, a page fault on the left child of $N_1^2$ triggers prefetching of the entire $N_1^2$ because `motion` bit of $N_1^2$ is 1 so both its children are considered as one combined leaf node. This in turn makes the $\mathcal{N}_{\text{migrated}}$ of $N_3^0$ greater than half of $\mathcal{N}_{\text{total}}$ of $N_3^0$ and causes all nodes under $N_3^0$ to be migrated. On the other hand, when a page fault occurs on the left child of $N_1^4$, only both children of $N_1^4$ will be migrated since $N_3^0$ and $N_3^1$ are

independent trees as `isolation` bit of $N_4^0$ is 1 and hence $\mathcal{N}_{\text{migrated}}$ of $N_4^0$ will be ignored by PE.

This tree configuration implicitly incurs page migrations to abide by the leaf node unit migrations. If the leaf size is increased and a subset of the enlarged leaf node is already in the GPU memory, we prefetch the remaining pages of the leaf node to the GPU memory upon tree configuration.

## 4.5 Eviction

As discussed in Section 3.3, the existing UVM page replacement policy incurs severe page thrashing. To mitigate the inefficiency, a study [27] proposed to use the page access counters for page eviction. As the original access counters reflect access frequency, they evict the least frequently used (LFU) blocks. However, LFU cannot resolve the page thrashing problem completely because recently accessed pages could be easily evicted due to limited access frequency history though they tend to have higher temporal locality.

To tackle the eviction inefficiency, we propose to leverage our repurposed access counters and implement a *pseudo-LRU*. As described in Section 4.2, the access counters in Forest hold the access recency information within each data object. Thus, we can identify the LRU page within each data object by finding the page with the smallest hardware counter register value.

To find the global LRU page, we leverage the *recency order* field in the ATT object table. The recency order field maintains the sorted order of access recency among the objects in the table. When data is accessed, the recency order field is updated such that the accessed data has the highest value of all (10 in our design as the object table has 10 entries). Therefore, the object with the smallest recency order (which is 1) is the LRU object. By leveraging that, we evict the leaf node that has the LRU page of the LRU object.

When GPU encounters a far-fault under memory oversubscription, ATT checks the object table and finds the LRU object. The LRU object information is sent with the far-fault interrupt to the UVM driver. Then, the UVM driver reads the page access counters of the LRU object (similar to the operations in Section 4.3), finds the LRU page, and commands eviction of the leaf node having the LRU page. Given that conventional LRU relies on page fault events and searches for the LRU page across the entire memory space, Forest confines its search to the LRU object, significantly reducing the search space and alleviating the associated overhead.

## 4.6 Driver Overhead

The UVM driver handles the pattern detection and tree configurations in parallel while the GPU continues the execution. Therefore, the driver operations do not affect the overall performance. The access counter transfer could incur performance overhead because it consumes the PCIe bandwidth. In the evaluation, we included the times for transferring the access counters for every profiling phase and the times for setting the cease bit upon pattern detection for every object. The times for tree traversal in Forest are similar or even less than the TBNp because NVIDIA driver uses 63 nodes per 2MB tree for TBNp, while four variant trees of Forest use 15 to 63 nodes, thus the depth of trees is equal to or shorter than in TBNp. The runtime overhead is evaluated in detail in Section 7.7.

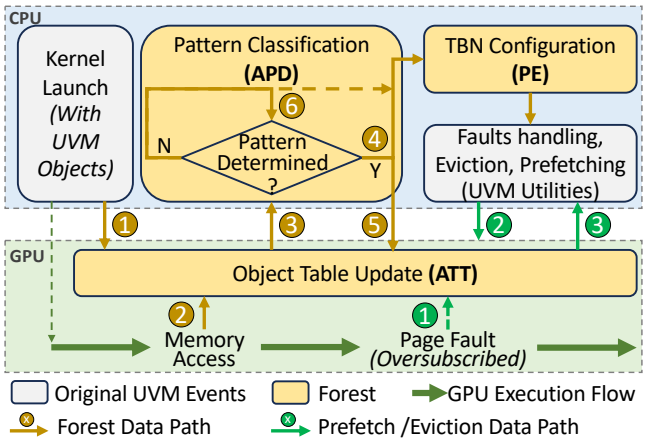



**Figure 11: Workflow of Forest.**

## 4.7 Bringing It All Together

The workflow of Forest is as follows and illustrated in Figure 11.

**Detecting access patterns and configuring trees:** Upon a kernel launch, the UVM driver first initializes the object table in ATT by populating the ATT entries for the objects accessed by the kernel. The UVM memory allocation APIs (e.g., `cudaMallocManaged`) will provide each object's information such as the VPN ranges (❶). During kernel execution, memory accesses update the access timer and recency order of the corresponding object in ATT, which is located in GMMU (❷). When an object has more than a predefined number of accesses (10K transactions in Forest), ATT triggers an interrupt to notify the GPU driver. The driver then retrieves the object's access timer information by using the driver's *existing* access counter copy function (❸). Then, the driver checks the access pattern via APD. Once the pattern is determined, the driver records it to the pattern table, reconfigures the object's tree by setting motion and isolation bits of the tree nodes in PE (❹), and notifies ATT to stop monitoring patterns for the object by setting the cease bit in the object table (❺). If a pattern is not detected after multiple detection attempts (e.g., 10 attempts in Forest), Forest marks the corresponding object as default (LC) pattern (❻). *Throughout this process, GPU does not need to stall its execution.*

**Prefetching with newly configured tree & eviction:** The reconfigured tree is then utilized when a new page fault occurs (❶). Once the fault is resolved, the driver prefetches data based on the newly configured tree (❷). *These steps are identical to the baseline UVM, except for using a new tree configuration.* When a page fault occurs under oversubscription, eviction is required. Then, Forest identifies the LRU object in the ATT's object table, selects the corresponding leaf node with the LRU page (i.e., the page with the smallest LRU timer), and proceeds with eviction (❸).

# 5 Speculative Forest

Forest uses profiling to detect the access patterns. To reduce the profiling phase and configure the optimal TBNp early, we propose *Speculative Forest (SpecForest)*, which uses pattern recording, static analysis, and access similarity detection.

**Table 1: Workloads and Access Patterns of UVM-managed Data Objects.**

| Category | App Name | Abbr. | Kernel | Data Object (Pattern) | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| Linear | 2D Conv [26] | 2DC | Convolution_kernel | A: LS | B: LS | | | | |
| | 3D Conv [30] | 3DC | Convolution3D_kernel | A: LS | B: LS | | | | |
| | AddVector [26] | AV | vecAdd | A: LS | B: LS | | | | |
| | FDTD-2D [26] | FDTD | fdtd_step1_kernel | _fict_: LS | ey: LS | hz: LS | | | |
| | | | fdtd_step2_kernel | ex: LS | hz: LS | | | | |
| | | | fdtd_step3_kernel | ex: LS | ey: LS | hz: LS | | | |
| | srad_v2 [26] | SRAD | srad_cuda_1 | E_C: LS | W_C: LS | N_C: LS | S_C: LS | J_cuda: LS | C_cuda: LS |
| | | | srad_cuda_1 | E_C: LS | W_C: LS | N_C: LS | S_C: LS | J_cuda: LS | C_cuda: LS |
| | stencil [26] | STEN | block2D_hybrid_... | A0: LS | Anext: LS | | | | |
| | hotspot [26] | HS | calculate_temp | power: LS | temp_src: LS | temp_dst: LS | | | |
| Mixed | dwt2d [30] | DWT | c_CopySrcToC... | d_r: LS | d_g: LS | d_b: LS | d_src: LS | | |
| | | | fdwt53Kernel1 | in: HCHI | out: HCHI | | | | |
| | | | fdwt53Kernel2 | in: LC | out: LC | | | | |
| | CifarNet [36] | CN | ExecuteThirdLayer | Layer3_W...: LS | Layer3_F...: LS | Layer2_pool: LS | | | |
| | | | ExecuteFourthLayer | Layer4_W...: HCHI | Layer4_F...: LS | Pool3_L...: HCHI | | | |
| | ra [26] | RA | kernel | input: LS | output: HCHI | table: HCHI | | | |
| | BICG [30] | BICG | bicg_kernel1 | A_gpu: LS | r_gpu: LS | s_gpu: LS | | | |
| | | | bicg_kernel2 | A_gpu: HCHI | p_gpu: LS | q_gpu: LS | | | |
| | nw [26] | NW | needle_cuda_shared_1 | reference: LC | itemsets: LC | | | | |
| | | | needle_cuda_shared_2 | reference: HCLI | itemsets: HCLI | | | | |
| | sssp [26] | SSSP | initializeArrays | shortest-Distances: LS | updating Short...: LS | | | | |
| | | | Kernel1 | vertexArray: LS | edgeArray: HCLI | weightArray: HCLI | shortest-Distances: LS | updating Short...: HCLI | |
| | | | Kernel2 | shortest-Distances: LS | updating Short...: LS | | | | |
| | bfs [26] | BFS | Kernel1 | g_graph_nodes: LS | g_graph_edges: LS | g_graph_mask: HCHI | g_updating_graph_mask: HCHI | g_graph_visited: LS | g_cost: HCHI |
| | | | Kernel2 | g_graph_mask: LS | g_updating_graph_mask: LS | g_graph_visited: LS | | | |
| | SqeezeNet [36] | SN | ExecuteFirstLayer | Weights: HCHI | Data_..._R: LS | Data_..._G: LS | Data_..._B: LS | Features: HCHI | |
| | | | pooling1 | Features: HCHI | Pool_layer: HCLI | | | | |
| | | | Executefire2expand1x1 | Weights: HCHI | Features: LS | Pool_layer: LS | | | |
| | | | Executefire2expand3x3 | Weights: HCHI | fire2_Features: LS | 1x1_Features: HCHI | | | |
| | | | global_pooling | Features: LS | output_GPU: LS | | | | |

*: Repeated kernel invocation instances only present the first one, since all the instances exhibit a similar pattern.

## 5.1 Access Pattern Recording

We leverage repetitive kernel executions to skip profiling. As described in Section 4.3, APD records the access patterns in the pattern table. We maintain the pattern information throughout the application execution. If an application runs a kernel multiple times, the recorded pattern is used without reprofiling the kernel. Such repetitive kernel execution can be easily found in large GPU applications because large data should be processed over multiple kernel executions by utilizing the limited hardware resources. When a kernel is launched, the APD checks if the kernel has the record in the pattern table. If so, the cease bits of the corresponding entries in the ATT's object table will be set and ATT will not trigger interrupts for profiling. APD will share the optimal tree configuration for the recorded pattern with PE and PE will configure the TBNp from the beginning of the execution.

```
1 __global__ void vecAdd(int n, float *x, float *y)
2 {
3   int index=blockIdx.x*blockDim.x+threadIdx.x;
4   int stride = blockDim.x * gridDim.x;
5   for (int i = index; i < n; i += stride)
6 ▶     y[i] = x[i] + y[i];
7 }
```

**Listing 1: A code snippet of AV.**

## 5.2 Static Access Pattern Detection

The pattern recording only works for kernels executing repetitively. For the kernels that do not have a recorded pattern, we apply a compile-time access pattern analysis. In GPU applications, data access patterns can be estimated by checking the data indexes [5, 60, 63]. Out of the four access patterns, static analysis can detect the LS pattern, which typically uses indexing with fixed strides. Listing 1 shows an example. Both `x` and `y` are accessed using the same index `i` (global thread id) with a fixed stride. Once the compiler detects LS data objects, the information is passed to `cudaMallocManaged(.)`. This one-bit information can be passed in many ways but we extend the existing API's parameter. `cudaMallocManaged(.)` uses `flags` parameter to indicate the access exclusiveness between CPU and GPU. It uses 1-bit information but we extend it to use 2 bits. If the upper bit of the `flags` is 1, it means the data object uses the LS pattern. Note that this pattern bit is set by the compiler. Thus, this doesn't require program modification. If the UVM driver finds this bit is set, LS pattern is recorded in the pattern table and the profiling is skipped.

```
1 __global__  void Kernel1(...) {
2   int tid = blockIdx.x*blockDim.x + threadIdx.x;
3   ...
4 ▶   int edgeStart = vertexArray[tid];
5 ▶   int edgeEnd =  vertexArray[tid + 1];
6   ...
7   for(int edge=edgeStart; edge<edgeEnd; edge++){
8 ▶     int nid = edgeArray[edge];
9 ▶     atomicMin(..., weightArray[edge]);
10 ... }}
```

**Listing 2: A code snippet of SSSP.**

### 5.3 Access Pattern Similarity Detection

The patterns HCHI, HCLI, and LC might include complex indexing (e.g., nested and indirect references). Therefore, the static analysis is not effective. To support these patterns, we propose a novel *access pattern similarity detection*. Listing 2 shows an example. In `SSSP`, both `edgeArray` and `weightArray` are indexed with the content of another array, `vertexArray`. Compilers cannot detect access patterns for such indirect indexing. Instead, we focus on the indexing similarity. As `edgeArray` and `weightArray` use the same index, `edge`, we can expect that they will follow the same access pattern. We group the two arrays into one pattern group.

During the index analysis, the compiler identifies UVM data objects that use the same index. If the index is an indirect reference or an outcome of a complex stride calculation, the compiler creates a similarity group and marks those data with the same group ID. We deliver the similarity group information via the same method that is used for static access pattern detection. We extend the `flags` parameter to have a few more bits to represent the group IDs. We use three bits because our benchmarks have up to eight similarity groups. If the UVM driver finds that the similarity bits in the `flags` parameter are non-zero, it encodes the similarity bit value (group ID) to the pattern field in APD's pattern table. To distinguish the similarity groups from the four access patterns, we add three bits to the pattern field; the most significant three bits are similarity group ID and the remaining two bits indicate the access patterns. If an object has non-zero values in the similarity group bits while the remaining two bits are zeros in the pattern field, the APD initiates the profiling. When one data's pattern is determined, APD searches the pattern table and marks the same pattern to the objects having the same similarity group ID. This way, we can configure TBN tree early and skip profiling for all objects in the same similarity group as soon as one object finishes the profiling.

### 5.4 SpecForest in Forest Workflow

SpecForest adds two steps on the Forest's workflow, Figure 11.

**Utilizing recorded patterns:** When a kernel is launched (❶), the driver checks whether the access patterns of each object have been recorded by searching the pattern table in the APD with the kernel name and the address range of the object. If a pattern is found, Forest configures the object's tree with the recorded pattern.

**Utilizing similarity:** When an object's pattern is determined (❹), SpecForest evaluates its pattern similarity, leveraging similarity detection results. If there are other objects in the same similarity group, their trees are reconfigured together in PE.

## 6 Discussions

**Why Tree?** Forest uses TBNp as the baseline prefetcher because it is used by all recent generations of NVIDIA GPUs and the tree-based prefetching is more efficient for bursty GPU memory behaviors and in controlling prefetch aggressiveness than conventional prefetching algorithms [1, 34, 35, 40, 62]. Note that the fundamental differences between CPU and GPU execution models hinder the adoption of advanced CPU-side prefetch algorithms to UVM [6, 32, 34, 61]. To support complicated access patterns, CPU solutions track individual load instruction-triggered cache misses while each load instruction can cause a storm of cache misses and faults in GPU. Detecting strides from such massive memory transactions is challenging. Furthermore, GPU hardware should be modified significantly to enable such instruction-level fine-grained monitoring. The problem becomes worse for UVM management because such fine-grained monitoring information should be delivered via slow PCIe and processed by CPU. Even for coarser-grained page faults, due to the storms of memory accesses, UVM drivers handle them in batch units to avoid significant CPU-side performance overhead [2, 8, 14]. The hierarchical tree structure can support such a unique GPU computing better because batches of leaf nodes can be concurrently handled. Beyond that, Forest enables a more flexible and autonomous (without geeky programmer's efforts) management with lightweight object-level access monitoring.

That being said, Forest can be also applied to non-tree prefetchers used by different GPUs. For example, AMD GPUs use range-based prefetching for shared virtual memory (SVM) [13]. SVM migrates memory in large units (Ranges) between 2MB and 1GB. Such an aggressive prefetching is effective in reducing far-faults, while its access-pattern obliviousness often results in significant memory thrashing [13]. Forest could resolve this problem by partitioning each Range to smaller migration units based on the access patterns of the associated object. Once AMD GPU supports per-page access count, ATT can be implemented with negligible overhead.

**Impact on emerging & diverse architectures** While we evaluated with discrete GPU systems, our idea can be applied to emerging and diverse architectures, such as Grace-Hopper (GH) superchips. GH uses cache-line-level communication between a Grace CPU and a Hopper GPU via a high-bandwidth interconnect [17, 57]. Similar to *zero-copy*, data can be shared via remote accesses rather than migration in GH. However, the impact of efficient prefetch cannot be underestimated. Due to the notable bandwidth gap among CPU local memory (500 GBps), inter-chip interconnect (900 GBps), and GPU local memory (4 TBps), pages with intensive GPU accesses are better migrated to GPU memory rather than shared remotely, thus needing efficient prefetch. Given that GPU's SIMD-like execution model inherently exhibits high page locality (our benchmarks access each page 3K times on average up to 14K times), Forest can manage memories of such emerging superchips efficiently. If GPU local memory is oversubscribed, the impact will be even higher.

## 7 Evaluation

### 7.1 Methodology

We extended a UVM-enabled GPGPU-Sim 4.0 [48] to design Forest. The simulator is integrated with UVM emulator including TBNp tree traversal function. We modified TBNp functions for Forest. The page fault handling latency and page table walk latency are set to 45 µs and 100 cycles, respectively [26, 27]. Remote memory access latency is set to 200 core cycles. The delayed migration threshold is set to 8 [27]. GPU memory is assumed to be oversubscribed by 150% by default. We considered the driver overhead as described in Section 4.6. Detailed simulator configurations are listed in Table 2.

*7.1.1* ***Compared solutions.*** We compare Forest with the following solutions including three state-of-the-art (SOTA) solutions.

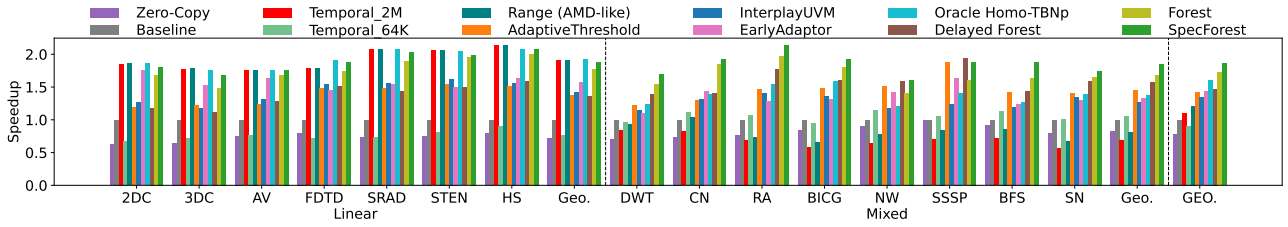



**Figure 12: Overall speedup over Baseline (higher is better).**

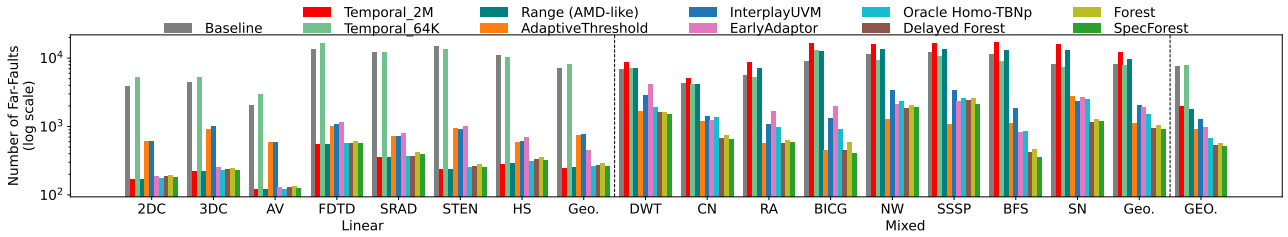



**Figure 13: Number of far-faults (lower is better): Note that Zero-Copy doesn't incur page faults.**

- **Baseline** [2, 26] is the TBNp that uses 2MB trees and 64KB leaf nodes. The UVM driver selects a 2MB VABlock from the LRU list, which is updated during far-faults and migrations.
- **Oracle Homo-TBNp** is an oracle application-level TBNp configuration. We assume that GPU knows the optimal homogeneous TBN tree for each application without profiling. We chose the configuration that performs best in Figure 4.
- **Zero-copy** uses cache-line-level remote memory accesses instead of page-level migration or prefetching [17, 57]. Host data is accessed without a page fault. Instead, a 128B cache line is copied from CPU to GPU [49, 54].
- **Delayed Forest** triggers migration or prefetching only when the pages are accessed more than a threshold. We set the threshold 8 as in Table 2.
- **Range** [13] is AMD's SVM approach. Range migrates and evicts data in variable units, at least 2MB. We configured to manage each object with one to multiple 2MB Ranges.
- **InterplayUVM** (SOTA) [26] extends TBNp with a tree-based eviction policy supporting locality-aware eviction, similar to proactive prefetching. If any node has more than half its child nodes on the host, all child nodes are evicted. Forest builds on this with a dynamic, tree-aware structure that maintains TBN tree semantics.
- **EarlyAdaptor** (SOTA) [29] uses the baseline TBNp and controls the prefetch aggressiveness by adapting the migration threshold. It also develops a model that dynamically adjusts prefetching intensity based on far-fault history.
- **AdaptiveThreshold** (SOTA) [27] adaptively alternates between remote zero-copy access to host-pinned memory and on-demand page migration. This adaptation leverages the existing hardware access counter.
- **TemporalPrefetcher** [34] applies temporal correlation-based prefetching. We implemented a correlation table tracking faulty block sequences. On a page fault, it migrates the faulty block with the next two correlated blocks recorded in the table [35]. We evaluated two faulty block granularities, 64KB (Temporal_64K) and 2MB (Temporal_2M).

**Table 2: Simulator Configuration.**

| | |
|---|---|
| **Simulator [48]** | GPGPU-Sim 4.0 (UVM supported) |
| **GPU Architecture** | NVIDIA Turing architecture |
| **GPU Cores** | 30 SMs, 64 cores each @ 1365 MHz |
| **Page Size** | 4KB |
| **Far-Fault Handling Latency [26]** | 45 $\mu$s |
| **Page Table Walk Latency [26]** | 100 core cycles |
| **CPU-GPU Interconnect** | PCIe 3.0 x16 (8 GT/s/lane) |
| **Access Counter Threshold [27]** | 8 |
| **Remote Access Latency [27]** | 200 core cycles |

*7.1.2* ***Workloads.*** We evaluated Forest with 15 workloads selected from widely used UVM benchmarks, UVMBench [30] and InterplayUVM [26]. We also tested CNN workloads in Tango [36] by replacing `cudaMalloc` API calls with `cudaMallocManaged`. Table 1 shows the detailed patterns of various data objects across kernels. These workloads use a working set size from 19.5MB to 144MB, with an average memory footprint of 63.5 MB. By adjusting the memory oversubscription ratio, we manipulate the device memory size instead of changing the benchmarks' working set sizes. In Section 7.6, we also assessed a variety of real-world deep learning models, AlexNet [39], ResNet50 [31], Google's BERT [20], and OpenAI Whisper [52], running within PyTorch framework. These models have footprints from 226MB to 891MB. For these models, we integrated the UVM-enabled GPGPU-Sim with Accel-Sim [37].

## 7.2 Overall Performance & Far-fault Reduction

Figure 12 shows the overall speedup over the Baseline. Forest shows superior performance with average speedups ranged from 1.07× to 2.21× compared with the solutions described in Section 7.1.1 (1.72× over Baseline, up to 1.39× over SOTA, and up to 1.9× over TemporalPrefetcher). SpecForest attains over 8% further improvement

than Forest (1.86× over Baseline) because SpecForest can configure the ideal TBNp earlier than Forest.

To understand the speedup, we measured the number of far-faults as shown in Figure 13. A strong correlation is observed between the performance and the total number of far-faults. Forest especially showed better far-fault reduction for linear applications compared to AdaptiveThreshold and EarlyAdaptor because the migration aggressiveness of these solutions is limited to the 2MB unit, which causes more far-faults when applications access over the 2MB address boundary. Forest reduces such far-faults by employing a larger tree. The significant far-fault reduction of AdaptiveThreshold for mixed pattern applications didn't lead to better performance because AdaptiveThreshold replaces far-faults with slow remote accesses. On the other hand, SpecForest reduces far-faults comparably with AdaptiveThreshold for mixed-pattern applications, and such reduction is well reflected in the performance because the smaller tree and leaf nodes for HCHI and HCLI objects and the proposed pseudo-LRU reduce page thrashing without incurring slow remote accesses. Forest outperforms Oracle Homo-TBNp significantly for the mixed-pattern applications because Oracle Homo-TBNp uses one tree configuration for all data objects. Though the tree might be the best among different configurations, the configuration might not be best for some data objects. Zero-Copy consistently exhibits poor performance due to the slow remote memory accesses and its inability to utilize the high-bandwidth device memory or exploit the locality of GPU applications. Range shows excellent performance for LS patterns because of its aggressive prefetching. However, its performance significantly degrades in mixed-pattern applications because of severe page thrashing, which we believe Forest's access-aware prefetching and gradual migration aggressiveness could solve. Delayed Forest shows almost comparable performance with Forest for mixed-pattern workloads because it prevents unnecessary migrations of sparsely accessed pages. However, it also delays effective migrations for LS workloads, which leads to over 17% worse performance than SpecForest for LS workloads. TemporalPrefetcher prefetches blocks based on faulty block history, making it efficient for complex yet repeated accesses. For LS patterns that barely have repeated accesses (e.g., streaming), correlation prefetching does not bring significant speedup. Instead, the size of each migration unit (2MB in Temporal_2M and 64KB in Temporal_64K) implicitly supports prefetching. Thus, Temporal_2M shows a good performance for Linear benchmarks since each page fault triggers a migration of 2MB, similar to Range. On the other hand, it shows the worst performance of all for Mixed benchmarks because the aggressive prefetching causes severe GPU memory thrashing, thereby diminishing the impact of correlations. Temporal_64K shows a better performance for the Mixed benchmarks. However, it still lags behind other GPU prefetchers because it supports limited locality (within 64KB block) for LS patterns and its sequence-based design lacks support for bursty access behaviors of GPU computing.

## 7.3 Page Thrashing and Eviction Performance

We measured the number of page thrashing events while running SpecForest. To understand the impact of optimal tree configuration and on-device eviction separately, Figure 14 shows the reduced page thrashings with each of them separately. We enabled only optimal

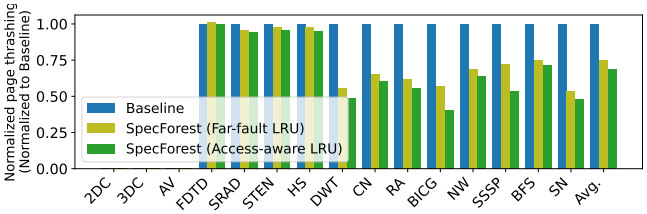



**Figure 14: Page thrashing normalized by Baseline.**

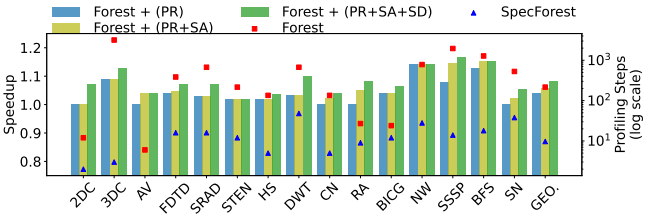



**Figure 15: Speedup breakdown (bar charts) and the profiling steps (scatter plot) of SpecForest.**

tree configuration while using default UVM eviction policy in the *SpecForest (Far-fault LRU)* measurements. *SpecForest (Access-aware LRU)* is the full integration of our proposed ideas. On average, the proposed tree configuration reduces page thrashings by 25%. The access-aware page eviction further reduces thrashings by 7% thanks to a more precise LRU block selection.

## 7.4 Performance Breakdown of SpecForest

We investigated the effectiveness of SpecForest on profiling step reduction. As shown with the scatter plot in Figure 15, SpecForest reduces profiling steps from 223 to 10 steps on average. The SpecForest employs pattern recording, static analysis, and similarity detection, and the bar charts of Figure 15 show the performance impact of each of them. The results indicate that pattern recording (denoted PR) offers more benefits for mixed-pattern benchmarks. This is because recording can save tree reconfiguration overhead more in mixed pattern applications. Static analysis (SA) and similarity detection (SD) each add 2% further speedups on average.

**Table 3: GPU configurations**

| GPU | Arch. | Mem. | Cores | H2D Interconn. |
|---|---|---|---|---|
| GTX 1080Ti [26] | Pascal | GDDR5X | 28 SMs<br>128 cores/SM<br>1481 MHz | PCIe 3.0 x16 |
| Quadro V100 [59] | Volta | HBM2 | 80 SMs<br>64 cores/SM<br>1132 MHz | PCIe 3.0 x16 |
| RTX 2060 [55] | Turing | GDDR6 | 30 SMs<br>64 cores/SM<br>1365 MHz | PCIe 3.0 x16 |
| RTX 3070 [58] | Ampere | GDDR6 | 46 SMs<br>128 cores/SM<br>1320 MHz | PCIe 4.0 x16 |
| H100 [15] | Hopper | HBM2e | 114 SMs<br>128 cores/SM<br>1095 MHz | PCIe 5.0 x16 |

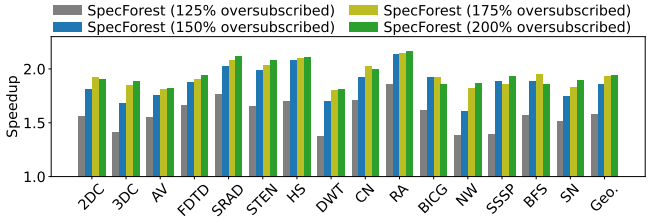



**Figure 16: Speedup under diverse oversubscription.**

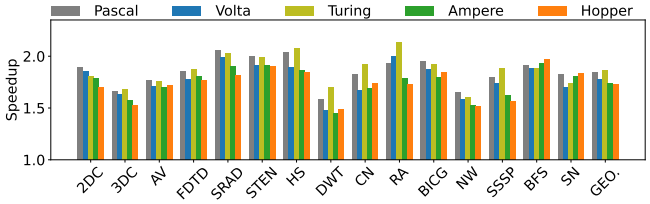



**Figure 17: Speedup of SpecForest on different GPUs.**

## 7.5 Sensitivity Study

*7.5.1 Sensitivity on Oversubscriptions.* We evaluated the impact of subscription ratios. As shown in Figure 16, in all oversubscription ratios, SpecForest consistently achieves notable speedups from 1.57× to 1.95× over Baseline. The results show that our solution delivers superior performance at higher memory oversubscription ratios due to increased device-host data transfers, which exacerbate the inefficiencies in sub-optimal tree configurations. SpecForest can effectively tackle the problem.

*7.5.2 Sensitivity on GPU Architecture.* When testing on five different GPU architectures, as listed in Table 3, SpecForest consistently shows notable performance improvements as shown in Figure 17 since workloads' memory access pattern is the main optimization indicator of our approach, rather than specific instruction set architecture. For the latest H100, while lower core frequency and higher memory bandwidth can improve the performance of memory-bound workloads, almost tripled SM count increases concurrency, which leads to higher interconnect and memory contentions. Consequently, UVM overhead remains as a bottleneck, which is effectively resolved by SpecForest.

*7.5.3 Sensitivity on Pattern Classification Interval.* We evaluated the impact of the pattern classification interval by varying it from 1K to 100K memory transactions on Forest. As shown in Figure 18, a short interval (<5K) fails to detect patterns within 10 classification rounds, often resulting in an undetermined classification that defaults to the conventional tree (e.g., 1K for 2DC, 3DC, FDTD; 5K for 2DC). Conversely, a long interval (>100K) delays switching to the optimal tree, as observed in STEM, CN, and RA. In cases like NW, where kernels have limited memory accesses, pattern classification never occur, forcing to use default TBN tree.

*7.5.4 Sensitivity on Threshold of Access Coverage & Intensity.* We evaluated the impact of different access detection thresholds. First, for the access coverage thresholds, we tested from 0.2 to 0.8, while we used 0.6 in all other evaluations. Figure 19a shows that too

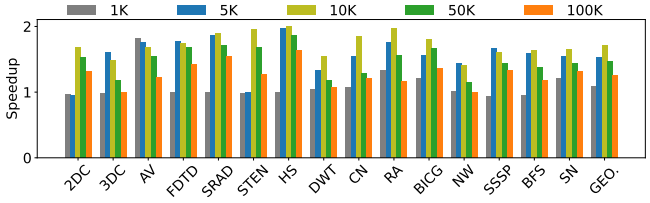
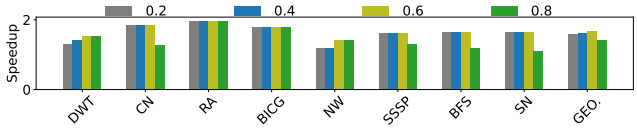
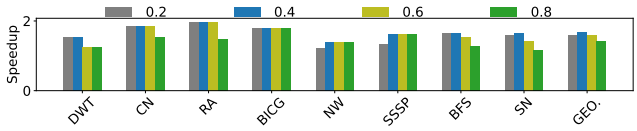



**Figure 18: Impact of pattern classification interval lengths (default 10K).**




**(a) Access coverage thresholds (default 0.6).**




**(b) Access intensity thresholds (default 0.4).**

**Figure 19: Impact of thresholds.**

high threshold misclassifies HC as LC (e.g., CN, SSSP, BFS, and SN). This misclassification fails to restrict access aggressiveness, leading to performance degradation. Conversely, too low threshold misclassifies LC as HC (e.g., DWT and NW), resulting in suboptimal tree configuration. We also tested with various access intensity thresholds from 0.2 to 0.8. We used 0.4 in all other evaluations. Figure 19b illustrates that too low threshold misclassifies HCLI as HCHI (e.g., SSSP and NW), while too high threshold misclassifies HCHI as HCLI (e.g., CN, RA, BFS, SN). These misclassifications hinder benchmarks from utilizing the appropriate tree configuration, impacting performance. These results verify our proper setting of default thresholds.

## 7.6 Forest on Real-world DL Workloads

We integrated Accel-Sim [37] with the UVM-enabled GPGPU-SIM to evaluate SpecForest for real-world deep-learning (DL) workloads. As shown in Figure 20, SpecForest achieves an average speedup of 1.51× (up to 1.62×). To understand these promising speedups, we measured the breakdown of access patterns, analyzing the distribution of various patterns as defined in Section 4.3.2. While LS pattern is predominant across all models, particularly in CNN models, the HCHI pattern constitutes a significant proportion in Transformer models (e.g., BERT and Whisper) due to the irregular dense memory access behaviors in their self-attention layers. By supporting flexible prefetching aggressiveness for individual tensor's access patterns, SpecForest is especially effective for the advanced ML models having more complex and diverse access patterns (e.g., HCHI), which are challenging to be supported by access-pattern oblivious TBNp.

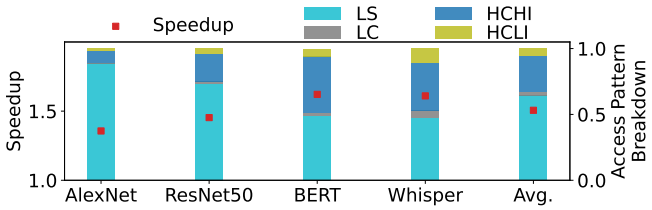



**Figure 20: Speedup and access pattern breakdown of SpecForest on real-world DL workloads.**

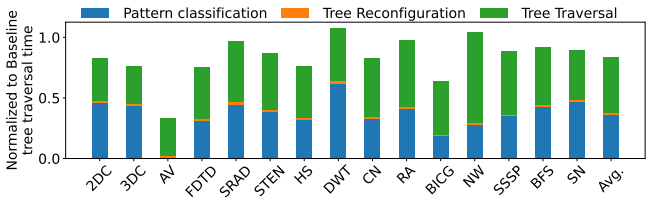



**Figure 21: Runtime overhead breakdown. (Normalized to the Baseline TBNp tree traversal time)**

## 7.7 Runtime Overhead

We implemented tree traversal, configuration, and pattern classification in the UVM driver emulator, which is integrated with the simulator. We measured the driver-side execution time when SpecForest is simulated on an Intel Xeon Gold 6330 server. Figure 21 shows the runtime overhead breakdown of SpecForest, normalized to the Baseline (TBNp) tree traversal time. Pattern classification occurs only at the beginning stage and ceases once the pattern is determined, making its impact negligible compared to the repeated tree traversal operations. Similarly, tree reconfiguration occurs only once after the pattern detection. Overall, the total runtime overhead of SpecForest is 16% less than the tree traversal time of TBNp. This is mainly contributed by the significantly reduced (by 54%) tree traversal time of SpecForest. As noted in Section 4.6, the proposed four trees have shorter or equal height with TBNp trees, thereby shorter traversal times for each search. Also, the reduced page thrashings and faults fundamentally decrease the traversal attempts.

## 7.8 Hardware Overhead

Forest leverages the existing UVM driver and page access counter architecture. The only addition to the hardware is ATT, which uses 147-byte per-kernel object tables. To update the recency order, it adds 4-bit comparators.

## 8 Conclusion

TBNp plays a significant role in the UVM performance. In this paper, we disclose intrinsic inefficiencies of conventional TBNp and propose Forest, which tackles the problems by employing access-aware heterogeneous TBNp configurations and a lightweight page eviction policy. The proposed software-hardware co-design allows dynamic TBNp adjustments, which enhances overall benchmark performance by 86% over the conventional TBNp and 39% over SOTA solutions, and 51% on real world DNN workloads, compared to the baseline.

## Acknowledgments

This work was supported by NSF grants, CCF-2114514 and CAREER-2341039. Part of this research was conducted using Pinnacles (NSF MRI, # 2019144) at the Cyber Infrastructure and Research Technologies (CIRT) at the University of California Merced.

## References

[1] Thomas Alexander and Gershon Kedem. 1996. Distributed Prefetch-buffer/Cache Design for High Performance Memory Systems. In *Proceedings of the 2nd IEEE Symposium on High-Performance Computer Architecture (HPCA '96)*. IEEE Computer Society, USA, 254.

[2] Tyler Allen and Rong Ge. 2021. Demystifying GPU UVM Cost with Deep Runtime and Workload Analysis. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 141–150. doi:10.1109/IPDPS49936.2021.00023

[3] Tyler Allen and Rong Ge. 2021. In-depth analyses of unified virtual memory system for GPU accelerated computing. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (St. Louis, Missouri) *(SC '21)*. Association for Computing Machinery, New York, NY, USA, Article 64, 15 pages. doi:10.1145/3458817.3480855

[4] AMD Corporation. Accessed June 2024. Radeons Next-generation Vega Architecture. https://en.wikichip.org/w/images/a/a1/vega-whitepaper.pdf.

[5] Grant Ayers, Heiner Litz, Christos Kozyrakis, and Parthasarathy Ranganathan. 2020. Classifying Memory Access Patterns for Prefetching *(ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 513–526. doi:10.1145/3373376.3378498

[6] Mohammad Bakhshalipour, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2018. Domino Temporal Data Prefetcher. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 131–142. doi:10.1109/HPCA.2018.00021

[7] Gloria B Barrett. 2000. The Coefficient of Determination: Understanding r squared and R squared. *The Mathematics Teacher* 93, 3 (2000), 230–234.

[8] Arkaprava Basu, Joseph L. Greathouse, Guru Venkataramani, and Jan Vesely. 2018. Interference from GPU System Service Requests. In *2018 IEEE International Symposium on Workload Characterization (IISWC)*.

[9] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. 2020. Yolov4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934* (2020).

[10] Chia-Hao Chang, Adithya Kumar, and Anand Sivasubramaniam. 2021. To move or not to move?: page migration for irregular applications in over-subscribed GPU memory systems with DynaMap. In *ACM International Conference on Systems and Storage*.

[11] Steven Chien, Ivy Peng, and Stefano Markidis. 2019. Performance Evaluation of Advanced Features in CUDA Unified Memory. In *2019 IEEE/ACM Workshop on Memory Centric High Performance Computing (MCHPC)*. 50–57. doi:10.1109/MCHPC49590.2019.00014

[12] Sangjin Choi, Taeksoo Kim, Jinwoo Jeong, Rachata Ausavarungnirun, Myeongjae Jeon, Youngjin Kwon, and Jeongseob Ahn. 2022. Memory harvesting in {Multi-GPU} systems with hierarchical unified virtual memory. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. USENIX Association, Carlsbad, CA, 625–638. https://www.usenix.org/conference/atc22/presentation/choi-sangjin

[13] Bennett Cooper, Thomas R. W. Scogland, and Rong Ge. 2024. Shared Virtual Memory: Its Design and Performance Implication for Diverse Applications. In *Proceedings of the International Conference on Supercomputing* (Kyoto, Japan). 25 – 37.

[14] NVIDIA Corporation. Accessed Feb 2025. UVM GPU non-replayable faults. https://github.com/NVIDIA/open-gpu-kernel-modules/blob/main/kernel-open/nvidia-uvm/uvm_gpu_non_replayable_faults.c.

[15] NVIDIA Corporation. Accessed February 2025. NVIDIA H100 Tensor Core GPU Architecture. https://resources.nvidia.com/en-us-data-center-overview/gtc22-whitepaper-hopper.

[16] NVIDIA Corporation. Accessed June 2024. NVIDIA Pascal Architecture. https://www.nvidia.com/en-us/data-center/pascal-gpu-architecture/.

[17] NVIDIA Corporation. Accessed November 2024. NVIDIA Grace Hopper Superchip Architecture Whitepaper. https://resources.nvidia.com/en-us-grace-cpu/nvidia-grace-hopper.

[18] NVIDIA Corporation. Accessed November 2024. Tree-based Prefetcher for ATS (Grace Hopper). https://github.com/NVIDIA/open-gpu-kernel-modules/blob/main/kernel-open/nvidia-uvm/uvm_ats_faults.c#L381-L464.

[19] NVIDIA Corporation. Accessed October 2024. Table of Technical Specifications per Compute Capability. https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#features-and-technical-specifications-technical-specifications-per-compute-capability.

[20] Jacob Devlin. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[21] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *CoRR* abs/1810.04805 (2018). arXiv:1810.04805 http://arxiv.org/abs/1810.04805

[22] Xianzhong Ding, Yunkai Zhang, Binbin Chen, Donghao Ying, Tieying Zhang, Jianjun Chen, Lei Zhang, Alberto Cerpa, and Wan Du. 2023. Vmr2l: Virtual machines rescheduling using reinforcement learning in data centers.

[23] Xianzhong Ding, Yunkai Zhang, Binbin Chen, Donghao Ying, Tieying Zhang, Jianjun Chen, Lei Zhang, Alberto Cerpa, and Wan Du. 2025. Towards VM Rescheduling Optimization Through Deep Reinforcement Learning. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) *(EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 686–701. doi:10.1145/3689031.3717476

[24] Yuan Feng, Seonjin Na, Hyesoon Kim, and Hyeran Jeon. 2024. Barre Chord: Efficient Virtual Memory Translation for Multi-Chip-Module GPUs. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 834–847. doi:10.1109/ISCA59077.2024.00065

[25] Debashis Ganguly, Rami Melhem, and Jun Yang. 2021. An Adaptive Framework for Oversubscription Management in CPU-GPU Unified Memory. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1212–1217. doi:10.23919/DATE51398.2021.9473982

[26] Debashis Ganguly, Ziyu Zhang, Jun Yang, and Rami Melhem. 2019. Interplay between hardware prefetcher and page eviction policy in CPU-GPU unified virtual memory. In *Proceedings of the 46th International Symposium on Computer Architecture* (, Phoenix, Arizona,) *(ISCA '19)*. Association for Computing Machinery, New York, NY, USA, 224–235. doi:10.1145/3307650.3322224

[27] Debashis Ganguly, Ziyu Zhang, Jun Yang, and Rami Melhem. 2020. Adaptive Page Migration for Irregular Data-intensive Applications under GPU Memory Oversubscription. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 451–461. doi:10.1109/IPDPS47924.2020.00054

[28] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. *IEEE Micro* 44, 3 (2024), 33–39. doi:10.1109/MM.2024.3373763

[29] Seokjin Go, Hyunwuk Lee, Junsung Kim, Jiwon Lee, Myung Kuk Yoon, and Won Woo Ro. 2023. Early-Adaptor: An Adaptive Framework forProactive UVM Memory Management. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 248–258. doi:10.1109/ISPASS57527.2023.00032

[30] Yongbin Gu, Wenxuan Wu, Yunfan Li, and Lizhong Chen. 2020. Uvmbench: A comprehensive benchmark suite for researching unified virtual memory in gpus. *arXiv preprint arXiv:2007.09822* (2020).

[31] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778. doi:10.1109/CVPR.2016.90

[32] Akanksha Jain and Calvin Lin. 2013. Linearizing irregular memory accesses for improved correlated prefetching. In *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture* (Davis, California) *(MICRO-46)*. Association for Computing Machinery, New York, NY, USA, 247–259. doi:10.1145/2540708.2540730

[33] Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor. 2023. Linear regression. In *An introduction to statistical learning: With applications in python*. Springer, 69–134.

[34] Doug Joseph and Dirk Grunwald. 1997. Prefetching using Markov predictors. *SIGARCH Comput. Archit. News* 25, 2 (May 1997), 252–263. doi:10.1145/384286.264207

[35] Jaehoon Jung, Jinpyo Kim, and Jaejin Lee. 2023. DeepUM: Tensor Migration and Prefetching in Unified Memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 207–221. doi:10.1145/3575693.3575736

[36] Aajna Karki, Chethan Palangotu Keshava, Spoorthi Mysore Shivakumar, Joshua Skow, Goutam Madhukeshwar Hegde, and Hyeran Jeon. 2019. Tango: A Deep Neural Network Benchmark Suite for Various Accelerators. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 137–138. doi:10.1109/ISPASS.2019.00021

[37] Mahmoud Khairy, Zhesheng Shen, Tor M. Aamodt, and Timothy G. Rogers. 2020. Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 473–486. doi:10.1109/ISCA45697.2020.00047

[38] Hyojong Kim, Jaewoong Sim, Prasun Gera, Ramyad Hadidi, and Hyesoon Kim. 2020. Batch-Aware Unified Memory Management in GPUs for Irregular Workloads. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 1357–1370. doi:10.1145/3373376.3378529

[39] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. ImageNet classification with deep convolutional neural networks. (2012), 1097–1105.

[40] An-Chow Lai, Cem Fide, and Babak Falsafi. 2001. Dead-block prediction & dead-block correlating prefetchers. (2001), 144–154. doi:10.1145/379240.379259

[41] Bingyao Li, Yanan Guo, Yueqi Wang, Aamer Jaleel, Jun Yang, and Xulong Tang. 2023. IDYLL: Enhancing Page Translation in Multi-GPUs via Light Weight PTE Invalidations. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 1163–1177.

[42] Bingyao Li, Jieming Yin, Anup Holey, Youtao Zhang, Jun Yang, and Xulong Tang. 2023. Trans-FW: Short Circuiting Page Table Walk in Multi-GPU Systems via Remote Forwarding. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 456–470. doi:10.1109/HPCA56546.2023.10071054

[43] Bingyao Li, Jieming Yin, Youtao Zhang, and Xulong Tang. 2021. Improving address translation in multi-gpus via sharing and spilling aware tlb design. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 1154–1168.

[44] Chen Li, Rachata Ausavarungnirun, Christopher J. Rossbach, Youtao Zhang, Onur Mutlu, Yang Guo, and Jun Yang. 2019. A Framework for Memory Oversubscription Management in Graphics Processing Units. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) *(ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 49–63. doi:10.1145/3297858.3304044

[45] Lingda Li and Barbara Chapman. 2019. Compiler assisted hybrid implicit and explicit GPU memory management under unified address space. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Denver, Colorado) *(SC '19)*. Association for Computing Machinery, New York, NY, USA, Article 51, 16 pages. doi:10.1145/3295500.3356141

[46] Mao Lin and Hyeran Jeon. 2025. Understanding Oversubscribed Memory Management for Deep Learning Training. In *Proceedings of the 5th Workshop on Machine Learning and Systems* (World Trade Center, Rotterdam, Netherlands) *(EuroMLSys '25)*. Association for Computing Machinery, New York, NY, USA, 46–55. doi:10.1145/3721146.3721955

[47] Mao Lin, Keren Zhou, and Pengfei Su. 2023. DrGPUM: Guiding Memory Optimization for GPU-Accelerated Applications. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 164–178. doi:10.1145/3582016.3582044

[48] Yechen Liu, Timothy Rogers, and Clayton Hughes. 2022. Unified Memory: GPGPU-Sim/UVM Smart Integration. *SANDIA REPORT* (2022). https://www.osti.gov/servlets/purl/1844477/

[49] Seung Won Min, Kun Wu, Sitao Huang, Mert Hidayetoğlu, Jinjun Xiong, Eiman Ebrahimi, Deming Chen, and Wen-mei Hwu. 2021. Large graph convolutional network training with GPU-oriented data communication architecture. *Proc. VLDB Endow.* 14, 11 (July 2021), 2087–2100. doi:10.14778/3476249.3476264

[50] Saba Mostofi, Hajar Falahati, Negin Mahani, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2023. Snake: A Variable-length Chain-based Prefetching for GPUs. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (Toronto, ON, Canada) *(MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 728–741. doi:10.1145/3613424.3623782

[51] NVIDIA Corporation. Accessed June 2024. NVIDIA Linux open GPU kernel module source. https://github.com/NVIDIA/open-gpu-kernel-modules.

[52] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2023. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning* (Honolulu, Hawaii, USA) *(ICML'23)*. JMLR.org, Article 1182, 27 pages.

[53] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-infinity: breaking the GPU memory wall for extreme scale deep learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (St. Louis, Missouri) *(SC '21)*. Association for Computing Machinery, New York, NY, USA, Article 59, 14 pages. doi:10.1145/3458817.3476205

[54] Jie Ren, Dong Xu, Shuangyan Yang, Jiacheng Zhao, Zhicheng Li, Christian Navasca, Chenxi Wang, Harry Xu, and Dong Li. 2024. Enabling Large Dynamic Neural Network Training with Learning-based Memory Management. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 788–802. doi:10.1109/HPCA57654.2024.00066

[55] Tor Aamodt's research group. Accessed July 2024. GPGPU-sim distribution. https://github.com/gpgpu-sim/gpgpu-sim_distribution/blob/master/configs/tested-cfgs/SM75_RTX2060/gpgpusim.config.

[56] Nikolay Sakharnykh. Accessed June 2024. Unified memory on pascal and volta. https://on-demand.gputechconf.com/gtc/2017/presentation/s7285-nikolay-sakharnykh-unified-memory-on-pascal-and-volta.pdf.

[57] Gabin Schieffer, Jacob Wahlgren, Jie Ren, Jennifer Faj, and Ivy Peng. 2024. Harnessing Integrated CPU-GPU System Memory for HPC: a first look into Grace Hopper. *International Conference on Parallel Processing* (2024).

[58] Accel-Sim team. Accessed October 2024. GPGPU-sim distribution. https://github.com/accel-sim/gpgpu-sim_distribution/blob/dev-uvm/configs/tested-cfgs/SM86_RTX3070/gpgpusim.config.

[59] Accel-Sim team. Accessed October 2024. GPGPU-sim distribution (Accel-sim). https://github.com/accel-sim/gpgpu-sim_distribution/blob/dev-uvm/

configs/tested-cfgs/SM7_QV100/gpgpusim.config.

[60] Mark Weiser. 1984. Program slicing. *IEEE Transactions on software engineering* 4 (1984), 352–357.

[61] Hao Wu, Krishnendra Nathella, Dam Sunwoo, Akanksha Jain, and Calvin Lin. 2019. Efficient Metadata Management for Irregular Data Prefetching. In *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*. 1–13.

[62] Haoyang Zhang, Yirui Zhou, Yuqi Xue, Yiqi Liu, and Jian Huang. 2023. G10: Enabling An Efficient Unified GPU Memory and Storage Architecture with Smart Tensor Migrations. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (Toronto, ON, Canada) *(MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 395–410. doi:10.1145/3613424.3614309

[63] Yuxuan Zhang, Nathan Sobotka, Soyoon Park, Saba Jamilan, Tanvir Ahmed Khan, Baris Kasikci, Gilles A Pokam, Heiner Litz, and Joseph Devietti. 2024. RPG2: Robust Profile-Guided Runtime Prefetch Generation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 999–1013. doi:10.1145/3620665.3640396